

Affine Algorithm

IT4 Ibb UN

ENG: Nader AL-hoshaee

Affine Cipher Application

Plain Text:
hello

Cipher Text:
duppy

Decrypt Encrypt 3 Key 'a': 8 Key 'b':

Calculate Inverse for Key 'a' Inverse (mod 26) is: 9

```
import tkinter as tk
from tkinter import ttk, messagebox
```

```
#
=====
===
# Section 1: The original encryption logic (without Arabic)
#
=====
===
```

```
ENGLISH_ALPHABET_UPPER = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
ENGLISH_ALPHABET_LOWER = "abcdefghijklmnopqrstuvwxyz"
```

```
def gcd(a, b):
    """Calculates the Greatest Common Divisor (GCD)."""
    while b != 0:
        temp = b
        b = a % b
        a = temp
    return a

def mod_inverse(a, m):
    """
    Calculates the modular multiplicative inverse.
    Returns 1 if no inverse is found (mimicking the original C# code).
    """
    a = a % m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return 1

def affine_cipher(text, a, b, is_encrypt):
    """
    The main function for encryption and decryption (C# direct
    translation).
    """
    result = ""
    for char in text:
        # 1. Handle uppercase English letters
        if char in ENGLISH_ALPHABET_UPPER:
            m = len(ENGLISH_ALPHABET_UPPER)
            x = ENGLISH_ALPHABET_UPPER.find(char)
            if is_encrypt:
                transformed_char_index = (a * x + b) % m
            else: # Decryption
                a_inv = mod_inverse(a, m)
                transformed_char_index = (a_inv * (x - b)) % m
                if transformed_char_index < 0:
                    transformed_char_index += m
            result += ENGLISH_ALPHABET_UPPER[transformed_char_index]

        # 2. Handle lowercase English letters
        elif char in ENGLISH_ALPHABET_LOWER:
```

```

        m = len(ENGLISH_ALPHABET_LOWER)
        x = ENGLISH_ALPHABET_LOWER.find(char)
        if is_encrypt:
            transformed_char_index = (a * x + b) % m
        else: # Decryption
            a_inv = mod_inverse(a, m)
            transformed_char_index = (a_inv * (x - b)) % m
            if transformed_char_index < 0:
                transformed_char_index += m
        result += ENGLISH_ALPHABET_LOWER[transformed_char_index]

    # 3. If the character is not in any alphabet
    else:
        result += char
    return result

#
=====
===
# Section 2: The GUI class using Tkinter
#
=====
===

class AffineCipherApp(tk.Tk):
    def __init__(self):
        super().__init__()

        # Main window settings
        self.title("Affine Cipher Application")
        self.geometry("550x450")
        self.resizable(False, False)

        # Main frame for content organization
        main_frame = ttk.Frame(self, padding="10")
        main_frame.pack(fill="both", expand=True)

        # --- Create UI Elements ---

        # 1. Plain Text field
        ttk.Label(main_frame, text="Plain Text:", font=("Arial",
12)).pack(anchor="w", padx=5)
        self.plaintext_entry = tk.Text(main_frame, height=5, width=60,
font=("Arial", 12))
        self.plaintext_entry.pack(fill="x", padx=5, pady=(0, 10))

```

```

# 2. Cipher Text field
ttk.Label(main_frame, text="Cipher Text:", font=("Arial",
12)).pack(anchor="w", padx=5)
self.ciphertext_entry = tk.Text(main_frame, height=5, width=60,
font=("Arial", 12))
self.ciphertext_entry.pack(fill="x", padx=5, pady=(0, 10))

# Frame for keys and buttons
controls_frame = ttk.Frame(main_frame)
controls_frame.pack(fill="x", pady=10)

# 3. Key input fields
ttk.Label(controls_frame, text="Key 'b':", font=("Arial",
12)).pack(side="right", padx=(0, 5))
self.key_b_entry = ttk.Entry(controls_frame, width=5,
font=("Arial", 12), justify='center')
self.key_b_entry.pack(side="right", padx=(0, 10))

ttk.Label(controls_frame, text="Key 'a':", font=("Arial",
12)).pack(side="right", padx=(0, 5))
self.key_a_entry = ttk.Entry(controls_frame, width=5,
font=("Arial", 12), justify='center')
self.key_a_entry.pack(side="right", padx=(0, 20))

# 4. Control buttons
self.decrypt_button = ttk.Button(controls_frame, text="Decrypt",
command=self.decrypt_click)
self.decrypt_button.pack(side="left", padx=5)

self.encrypt_button = ttk.Button(controls_frame, text="Encrypt",
command=self.encrypt_click)
self.encrypt_button.pack(side="left", padx=5)

# 5. Inverse calculation button and result label
ttk.Separator(main_frame, orient='horizontal').pack(fill='x',
pady=10)
inverse_frame = ttk.Frame(main_frame)
inverse_frame.pack(fill="x")

self.inverse_result_label = ttk.Label(inverse_frame, text="Inverse
result will be shown here", font=("Arial", 10))
self.inverse_result_label.pack(side="right", padx=10)

```

```

        self.inverse_button = ttk.Button(inverse_frame, text="Calculate
Inverse for Key 'a'", command=self.calculate_inverse_click)
        self.inverse_button.pack(side="left", padx=5)

    def get_keys(self):
        """Helper function to get keys and validate they are integers."""
        try:
            a = int(self.key_a_entry.get())
            b = int(self.key_b_entry.get())
            return a, b
        except ValueError:
            messagebox.showerror("Input Error", "Please enter valid
integers for Key 'a' and Key 'b'.")
            return None, None

    def encrypt_click(self):
        """Encrypt button functionality."""
        a, b = self.get_keys()
        if a is None:
            return

        # The original check condition, now simplified for only English
alphabet
        if gcd(a, len(ENGLISH_ALPHABET_UPPER)) != 1:
            messagebox.showwarning("Invalid Key", f"Key 'a' ({a}) must be
coprime with 26.")
            return

        plain_text = self.plaintext_entry.get("1.0", "end-1c")
        encrypted_text = affine_cipher(plain_text, a, b, True)

        self.ciphertext_entry.delete("1.0", tk.END)
        self.ciphertext_entry.insert("1.0", encrypted_text)

    def decrypt_click(self):
        """Decrypt button functionality (mimicking the original logical
error)."""
        a, b = self.get_keys()
        if a is None:
            return

        if gcd(a, len(ENGLISH_ALPHABET_UPPER)) != 1:
            messagebox.showwarning("Invalid Key", f"Key 'a' ({a}) must be
coprime with 26.")
            return

```

```

        # **Important Note**: Mimicking the original C# code's logical
error
        # It decrypts the text from the "Plain Text" field, not the
"Cipher Text" field.
        text_to_decrypt = self.plaintext_entry.get("1.0", "end-1c")

        decrypted_text = affine_cipher(text_to_decrypt, a, b, False)

        # The result is placed in the Cipher Text field (as in the
original code)
        self.ciphertext_entry.delete("1.0", tk.END)
        self.ciphertext_entry.insert("1.0", decrypted_text)

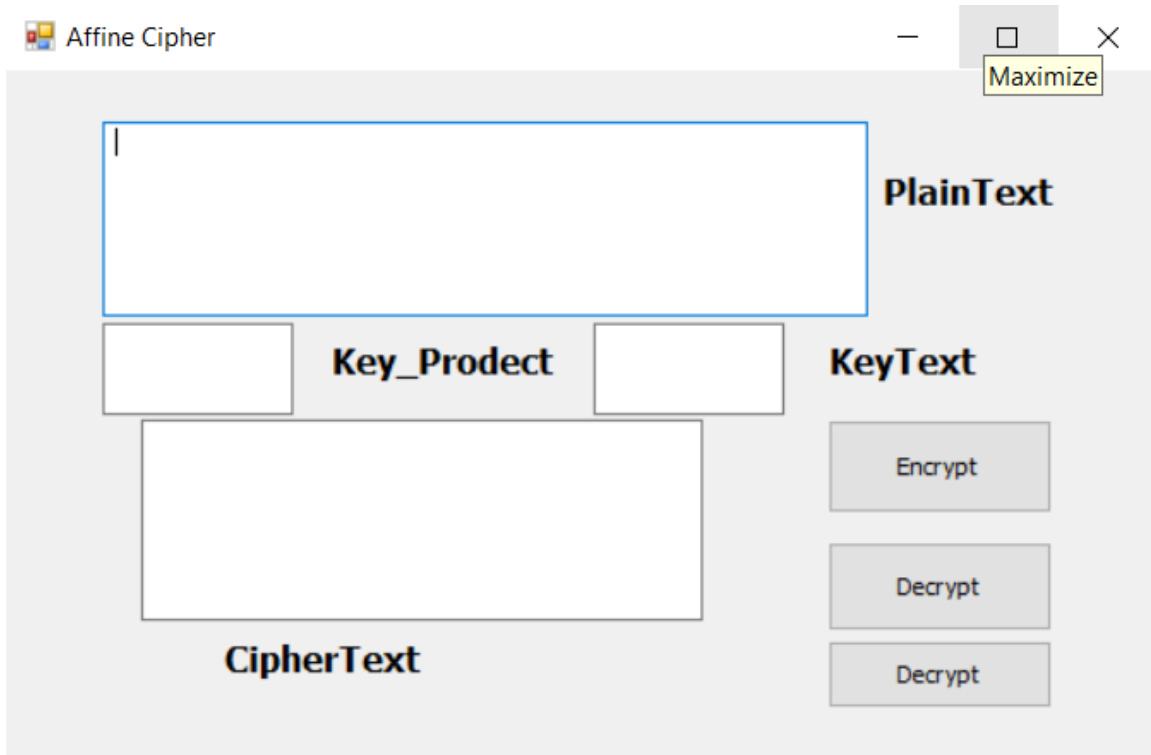
def calculate_inverse_click(self):
    """Inverse calculation button functionality."""
    try:
        a = int(self.key_a_entry.get())
        # Calculate inverse for the English alphabet (mod 26)
        alphabet_len = len(ENGLISH_ALPHABET_UPPER)
        inverse = mod_inverse(a, alphabet_len)

        # Since the original code returns 1 on failure, we'll clarify
the result
        if inverse == 1 and (a * 1) % alphabet_len != 1 and a != 1:
            result_text = "None"
        else:
            result_text = str(inverse)
        self.inverse_result_label.config(text=f"Inverse (mod 26) is:
{result_text}")

    except ValueError:
        messagebox.showerror("Input Error", "Please enter a valid
integer in the Key 'a' field.")

#
=====
====
# Section 3: Application entry point
#
=====
====
if __name__ == "__main__":
    app = AffineCipherApp()
    app.mainloop()

```



```
using System;
using System.Windows.Forms;

namespace AffineCipherApp
{
    public partial class Form1 : Form
    {
        private static string arabicAlphabet = "ابتثجحخدذرزسشصضطظعغفقكلمنهوي";
        private static string englishAlphabetUpper =
"ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        private static string englishAlphabetLower =
"abcdefghijklmnopqrstuvwxyz";

        public Form1()
        {
            InitializeComponent();
        }

        private static int GCD(int a, int b)
        {
            while (b != 0)
            {
                int temp = b; //3//1//2//1//1
                b = a % b; //3//1//2//1//1//0
            }
        }
    }
}
```

```

        a = temp;//4//3//1//2//1//1
    }
    return a;
}

private static int ModInverse(int a, int m)
{
    a = a % m;
    for (int x = 1; x < m; x++)
    {
        if ((a * x) % m == 1)
            return x;
    }
    return 1;
}

private string AffineCipher(string text, int a, int b, bool
isEncrypt)
{
    string result = "";
    foreach (char c in text)
    {
        if (arabicAlphabet.Contains(c.ToString()))
        {
            int m = arabicAlphabet.Length;
            int x = arabicAlphabet.IndexOf(c);
            int transformedCharIndex;
            if (isEncrypt)
            {
                transformedCharIndex = (a * x + b) % m;
            }
            else
            {
                int a_inv = ModInverse(a, m);
                transformedCharIndex = (a_inv * (x - b)) % m;
                if (transformedCharIndex < 0) transformedCharIndex
+= m;
            }
            result += arabicAlphabet[transformedCharIndex];
        }
        else if (englishAlphabetUpper.Contains(c.ToString()))
        {
            int m = englishAlphabetUpper.Length;
            int x = englishAlphabetUpper.IndexOf(c);
            int transformedCharIndex;
            if (isEncrypt)

```



```

        {
            transformedCharIndex = (a * x + b) % m;
        }
        else
        {
            int a_inv = ModInverse(a, m);
            Key_prodect.Text = a_inv + "";
            transformedCharIndex = (a_inv * (x - b)) % m;
            if (transformedCharIndex < 0) transformedCharIndex
+= m;

        }
        result += englishAlphabetUpper[transformedCharIndex];
    }
    else if (englishAlphabetLower.Contains(c.ToString()))
    {
        int m = englishAlphabetLower.Length;
        int x = englishAlphabetLower.IndexOf(c);
        int transformedCharIndex;
        if (isEncrypt)
        {
            transformedCharIndex = (a * x + b) % m;
        }
        else
        {
            int a_inv = ModInverse(a, m);
            transformedCharIndex = (a_inv * (x - b)) % m;
            if (transformedCharIndex < 0) transformedCharIndex
+= m;

        }
        result += englishAlphabetLower[transformedCharIndex];
    }
    else
    {
        result += c; //
    }
}

return result;
}

private void btnEncrypt_Click(object sender, EventArgs e)
{
    try
    {
        string plainText = txtPlainText.Text;

```

```

        int a = int.Parse(Key_prodect.Text);
        int b = int.Parse(key.Text);

        if (GCD(a, arabicAlphabet.Length) != 1 || GCD(a,
englishAlphabetUpper.Length) != 1 || GCD(a, englishAlphabetLower.Length)
!= 1)
        {
            MessageBox.Show("The key " + Key_prodect.Text + " and
the length of the alphabet must be coprime.");
            return;
        }

        string encryptedText = AffineCipher(plainText, a, b,
true);
        txtCipherText.Text = encryptedText;
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}

private void btnDecrypt_Click(object sender, EventArgs e)
{
    try
    {
        int a = int.Parse(Key_prodect.Text);
        int b = int.Parse(key.Text);

        if (GCD(a, arabicAlphabet.Length) != 1 || GCD(a,
englishAlphabetUpper.Length) != 1 || GCD(a, englishAlphabetLower.Length)
!= 1)
        {
            MessageBox.Show("The key 'a' and the length of the
alphabet must be coprime.");
            return;
        }
        txtCipherText.Text =
AffineCipher(txtPlainText.Text.Trim(), a, b, false);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}

```

```
private void Button3_Click(object sender, EventArgs e)
{
    try
    {
        label15.Text = ModInverse(int.Parse(Key_prodect.Text),
28).ToString();
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex+ "");
    }
}
}
```